

Project Proposal

Distributed Banking System using Microservices Architecture

1. Introduction

This project proposes a distributed banking system built using a microservices architecture. Instead of one large application, the system is divided into small independent services (Authentication, Customer, Account, Transaction, Notification, Audit, and Admin), each with its own database. This design allows the system to remain available, scalable, and able to handle heavy traffic without a single point of failure, while performing core banking operations such as deposits, withdrawals, and money transfers.

2. Objectives

- Design and implement a distributed, database-per-service banking system.
- Ensure data consistency across services using the Saga, Outbox, and Inbox patterns.
- Provide secure access using JWT-based authentication.
- Implement text and numerical search with sorting on transaction records.
- Route all client requests through a central API Gateway and Load Balancer.
- Demonstrate that the system remains stable and available under heavy simulated traffic.

3. Technology Stack

Frontend	Angular 20
Backend	ASP.NET Core (.NET)
Database	SQL Server (separate DB for each service)
Messaging	RabbitMQ
Cache	Redis
API Gateway	Ocelot / YARP
Authentication	JWT
Load Balancer	NGINX
Containerization	Docker
Monitoring	Prometheus + Grafana
Load Testing	JMeter / k6

4. Microservices Overview

4.1 Authentication Service

Handles register, login, logout and refresh token. Also handles change password and the forgot/reset password flow.

4.2 Customer Service

Manages the customer profile - create, update, view and delete. KYC document upload can be added here if time allows.

4.3 Account Service

Handles opening an account, viewing balance and account details, and freezing/unfreezing or closing an account.

4.4 Transaction Service

The main service for deposit, withdraw and transfer. Also handles transaction history, search and sort, and mini/full statement.

4.5 Notification Service

Sends email, dummy SMS and in-app notifications for important activity, and lets the user mark them as read.

4.6 Audit Service

Has no screen of its own - listens to events from other services and keeps a log of login, transaction and password-change activity for the admin to review.

4.7 Admin Service

A separate panel for the admin - login, dashboard, view customers/accounts/transactions, freeze/unfreeze accounts, and view audit logs.

5. Key Design Patterns

5.1 Saga Pattern

Used for the money transfer flow, which spans the Account and Transaction services with separate databases. The transfer is broken into small local transactions (debit sender, then credit receiver). If the credit step fails, a compensating transaction automatically refunds the sender, keeping the system consistent without a single distributed transaction.

5.2 Outbox Pattern

Used inside the Transaction Service and Account Service. When a service updates its database, it also writes the related event into an outbox table within the same local transaction. A background worker then publishes these events to RabbitMQ, so no event is lost even if the service crashes right after the update.

5.3 Inbox Pattern

Used inside the Notification Service and Audit Service. Incoming events from RabbitMQ are first recorded in an inbox table before being processed. This prevents the same event from being processed twice if RabbitMQ redelivers a message.

6. Handling Heavy Traffic

Each service can run as multiple instances behind the NGINX load balancer, so no single instance becomes a bottleneck. Redis caches frequently requested data such as balances and search results. Heavy traffic will be simulated using JMeter or k6 to generate a large number of concurrent requests, showing that the system continues to work correctly and does not go down under load.

7. RabbitMQ Events

- Customer Registered
- Account Created
- Money Deposited
- Money Withdrawn
- Money Transferred
- Password Changed
- Notification Sent

8. System Architecture Diagram

